

1. 自己紹介

1.1 使える技術

ゲーム開発スキル

種類	使用期間	レベル
Unreal Engine 5	約五カ月	本格的なゲームロジック設計・実装。責務分離設計・イベントキュー・マテリアル制御など C++クラス定義 + Blueprint拡張のハイブリッド開発 (Battery Collectorで実践) Run On Server / Multicast / Replicationを使った同期実装
C++	約一カ月	ゲーム開発スクールで習得 Rock Paper ScissorsをC++のみで実装 UE5との連携開発にも使用
Git / GitHub	約一カ月	個人開発でのバージョン管理・ソースコード管理

プログラミング言語

種類	使用期間	レベル
Python	約二年	大学院の研究で使用。数値計算・データ処理・表の出力
C言語	約半年	大学の講義で履修。基本的な文法・アルゴリズムを学習
HTML / CSS	約半年	大学の講義で履修。マイWebページの作成

1.2 これまでの開発経験

Unreal Engine 5を使った初めてのゲーム開発はYouTubeのチュートリアル動画を用いた独学で進めていました。

そこから書籍「Unreal Engine 5で極めるゲーム開発」を読破し、実装を重ねながら基本技術を習得しました。Blueprintによるゲームロジック構築・プレイヤー操作の実装・オブジェクト生成・UI 表示など、ゲームシステムの基礎を一通り学びました。

またゲーム開発のオンラインスクールにも通い、Rock Paper ScissorsやBattery Collectorなどの複数のゲーム制作を経験しました。これらの学習経験をもとに、オリジナル作品として3Dアクションパズルゲーム「Dice Puzzle」を制作しました。

1. 3 好きなゲーム

XI（サイ）

1998 年に SCE が発売したアクションパズルゲームです。プレイヤーがフィールド上のサイコロを押して同じ目を揃えて消していく面白さに小学生のころの私は強く惹かれたのを今でも鮮明に覚えています。後述の作品「Dice Puzzle」は XI に強くインスパイアされています。

クラッシュ・バンディクー3 ブッとび！世界一周

高難易度のステージをクリアしながら、隠し要素の発見やタイムアタックへの挑戦など、やり込み要素を徹底的に遊びました。シリーズは 2・4 作目もプレイ済みです。クリアするだけでは終わらない設計に夢中になりました。

サルゲッチュ

シリーズ 1～3 作目をすべてプレイしました。「見えているのに捕まえ方がわからない」という感覚が楽しく、試行錯誤しながら正解を見つけていく過程に熱中しました。プレイヤーに考えさせる設計の面白さを体感できた作品です。

モンスターハンター3rd

プレイ時間は 300 時間を超えました。モンスターを倒して装備を強化し、ドロップアイテムや装備をコンプリートしていく繰り返しの飽きることなく没頭しました。フレンドとのマルチプレイでは、シングルとは全く異なる楽しさがあることを実感しました。

2. 作品紹介 Dice Puzzle

2.1 ゲームの写真



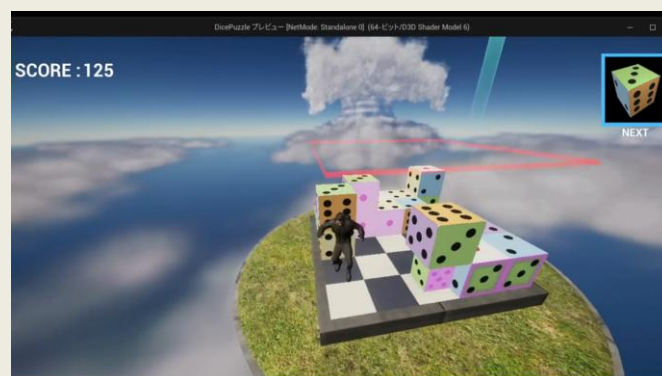
ゲームタイトル画面



ゲーム説明画面



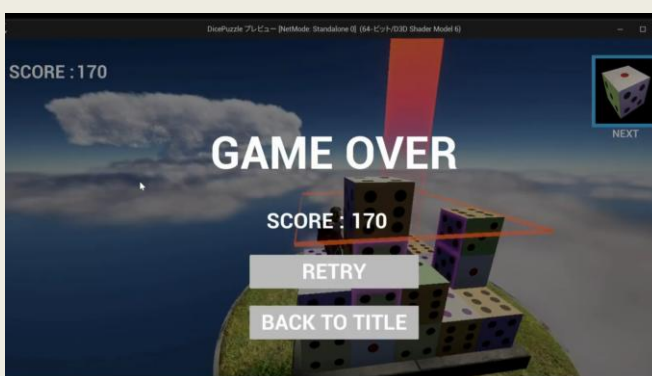
ダイスの消去直前



ダイスの消去直後



ゲームオーバー直前



ゲームオーバー画面

2.2 概要

ジャンル	3Dアクションパズル
開発期間	2026年2月20日 ～ 2026年3月13日
開発人数	個人開発
使用技術	Unreal Engine 5.2 / Blueprint
プレイ動画	https://youtu.be/OhvArU6PyAk
ソースコード	https://github.com/Tmak4263/DicePuzzle/tree/main

プレイヤーキャラクターがダイスの上に乗り、ダイスを移動させながら「同じ面・同じ目が3つ以上連結」した状態を作って消去するパズルゲームです。5×5グリッドの盤面にダイスがZ方向へ積み上がる3次元構造を持ち、定期的に上からダイスが落下するアクション要素が加わります。

2.3 ルール

- ・ 盤面：5×5グリッド（XY方向）。ダイスはZ方向に積み上がる
- ・ スポーン：9秒ごとに上からランダムな向きのダイスが落下してくる
- ・ 60秒ごとにスポーン間隔が1秒ずつ短縮し、最短3秒となる
- ・ 消去条件：同じ面の向き・同じ目の数字が3つ以上連結
- ・ スポーンによる消去は発生しない。プレイヤーがダイスを動かした後にのみ発動
- ・ 連鎖：プレイヤー操作による消去 → 落下 → 新たな一致が生まれた場合のみ連鎖
- ・ ゲームオーバー：ダイスが一定の高さ（Z=3ライン）を超えたら終了

2.4 操作説明

- ・ 移動：WASDキー、上下左右キー
- ・ 視点移動：マウスの移動
- ・ ジャンプ：スペースキー、マウスの左右クリック

2.5.1 Blueprint 責務分離設計

主要な 4 つの Blueprint

各 Blueprint はデータを直接読み書きせず、専用のインターフェースを通じて通信します。BoardManager は Actor を直接操作せず、BoardEventPlayer は OccupiedMap に触れません。この分離がデバッグのしやすさと将来拡張性の鍵となっています。

盤面は XYZ 整数座標で管理されます。ワールド座標とグリッド座標を相互変換する GridToWorld / WorldToGrid 関数を設計しました。

-

The screenshot shows a Pure Data patch for a grid-based movement simulation. The patch includes several key components:

- Grid to World:** A sub-component that receives input from a 'リターンノード' (Return Node) and outputs 'X', 'Y', and 'Z' coordinates.
- Cell Size:** A parameter input that influences the 'X', 'Y', and 'Z' coordinates.
- Make Vector:** A processing block that takes 'X', 'Y', and 'Z' coordinates and outputs a 'Return Value'.
- Board Origin:** A parameter input that influences the 'Return Value'.
- リターンノード (Return Node):** A sub-component that receives input from the 'Return Value' and outputs 'World Location'.

The patch is designed to calculate the 'World Location' based on the 'Grid to World' coordinates, 'Cell Size', and 'Board Origin'.

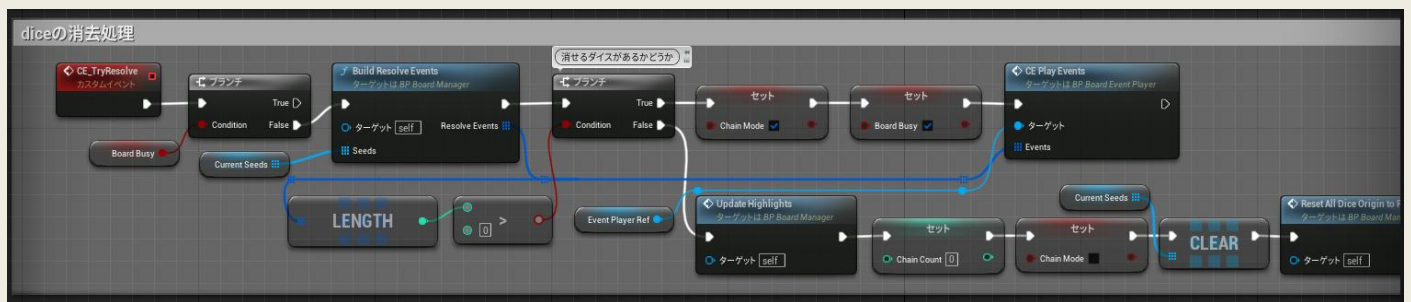
7

2.5.3 Seed 起点方式による消去判定アルゴリズム

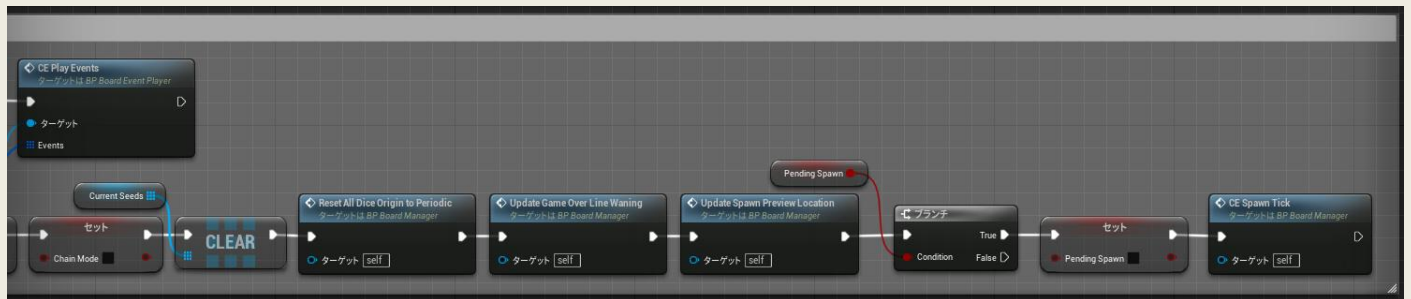
消去判定は「盤面全体を毎回走査する」方式ではなく、起点ダイス (Seed) から探索を始める Seed 起点方式を採用しています。

- ・ 初回消去の Seed: LastMoveDice (プレイヤーが最後に動かしたダイス)
- ・ 連鎖消去の Seed: ChainDrop 由来のダイス集合
- ・ Seed を含むグループが 3 つ以上一致した場合のみ消去発動。スポンで成立したグループは消えない
- ・ EDiceOrigin (Enum) で Periodic / PlayerMove / ChainDrop の由来を管理し、連鎖の起源を追跡

この設計により、「プレイヤーが意図的に動かした結果としてのみ消去・連鎖が起きる」という直感的なゲームルールを実現しています。



CE_TryResolve のノード構成 1



CE_TryResolve のノード構成 2

Build Resolve Events 内部で Seed 起点方式による盤面探索をしています。

2.5.4 イベントキュー方式によるアニメーション制御

消去・落下・スポンなどすべての演出は、FBoardEvent という Struct の配列としてキューに積まれ、BoardEventPlayer が Timeline で順番に再生します。

- ・ bBoardBusy フラグで再生中は新たなプレイヤー操作を受け付けない排他制御
- ・ OnQueueFinished Dispatcher でキュー完了を通知。これを受けて BoardManager がハイライト更新・次の処理を開始
- ・ 演出（見た目）とロジック（盤面状態）が完全に分離されているため、演出のみの変更がロジックに影響しない
- ・ フロー：プレイヤー操作
 - FBoardEvent[] 生成 (Slide/Remove/Drop/Spawn)
 - BoardEventPlayer がキューを順番に再生
 - OnQueueFinished
 - 次の処理へ

2.5.5 ダイス面ハイライトシステム

BP_Dice は M_FaceOverlay マテリアルと Dynamic Material Instance を使い、Highlight 状態変数 (0/1/2) で面ごとに3段階のハイライトを表現しています。

- ・ mode 0: ハイライトなし
- ・ mode 1: 枠強調 (2 連結の時)
- ・ mode 2: 全体発光 (3 連結以上の消去候補の時)
- ・ Opacity / Color / Intensity など視覚的なフィードバックを実現



2 連結の時のハイライト



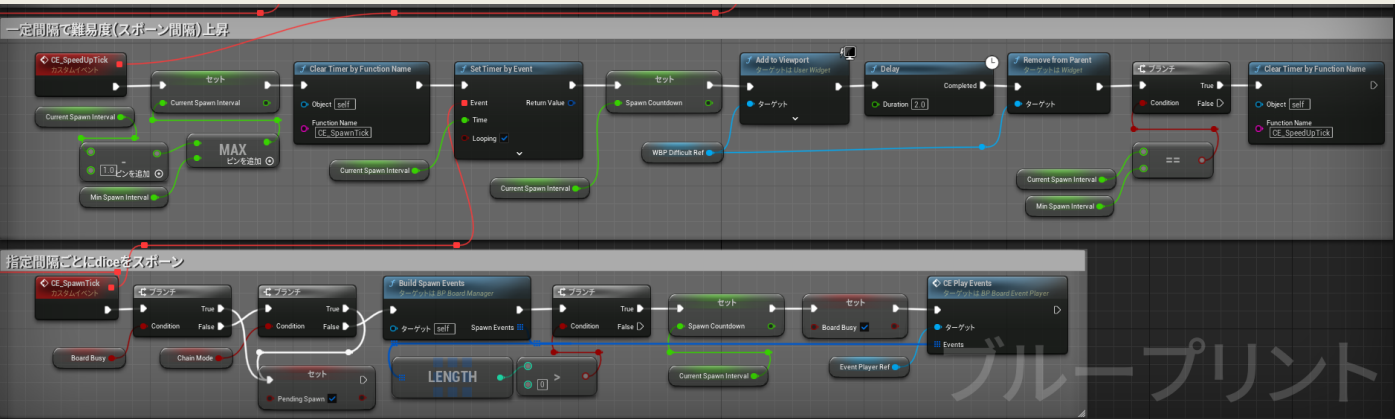
3 連結以上の時のハイライト

2.5.6 スポン加速システム

時間経過とともにスポン間隔を段階的に短縮する難易度上昇システムを実装しました。

- ・ 開始時のスポン間隔: 9 秒。60 秒ごとに CE_SpeedUpTick が発火し、間隔を 1 秒短縮
- ・ 最短 3 秒を下限として保証。最短到達後は CE_SpeedUpTick 自身を停止
- ・ タイマーのリセット方式: 既存の CE_SpawnTick タイマーを Clear → 新しい間隔で再 Set することで確実に反映
- ・ 連鎖中 (bChainMode = true) はスポンを一時停止。連鎖終了後に再開する排他制御も実装済み

この仕組みにより、序盤はゆっくり考えられる難易度から始まり、時間が経つほど判断スピードが求められるゲームテンポの変化を実現しています。



CE_SpeedUpTick と CE_SpawnTick

2.5.7 プレイヤー入力とキャラクター動作

スティック入力からダイスのスライドまでは、3段階の判定を順番に経る構造になっています。「歩く」と「ダイスを押す」を自然に使い分けられるよう、誤操作を防ぐ設計を意識しました。

主要な関数	
GetMoveDirFromInput	スティック入力をカメラのYaw 角基準で4方向 (PosX/NegX/PosY/NegY) に変換。カメラ向きに関わらず直感的な操作を実現
CanWalkInDirection	入力方向に隣のダイスがある場合は通常歩行を優先。ない場合のみホールド判定へ進む
IsTouchingEdgeInDirection	LineTrace でダイスの縁まで到達しているかを確認。到達している場合のみ HoldTime を加算し、誤操作を防止
UpdateHoldState	HoldTime を加算し、HoldThreshold 到達で1回だけスライドを確定
CE_RequestPlayerMove	スライド確定後に BoardManager へ移動要求を送信。BP_ThirdPersonCharacter は盤面データに直接触れず、要求を投げるだけ

またジャンプ入力はダイス上では「降りる」または「登る」動作として機能します。bIsOnDice フラグとコリジョン状態をリセットし、通常のジャンプ処理へ引き継ぎます。

- ・ 搭乗判定 (IsStandingOnDice 関数) は毎 Tick 実行。足元の LineTrace でダイスの上に立っているか常時確認
- ・ 隣にダイスがある方向はコリジョンを Block、ない方向は Ignore に動的切り替え (ApplyEdgeCollisionState 関数)。これにより隣ダイスへ歩いて渡れる動作を実現
- ・ 上からダイスがスポーンしてキャラクターが押しつぶされた場合は、ダイス上面へ自動救出テレポートする (ResolveCrushBySpawnedDice 関数)

2.6 苦労した点・工夫した点

最も苦労した点：キャラクターとダイスの移動処理の同期

本作では、プレイヤーキャラクターがダイスの上に乗った状態で端まで移動すると、ダイスが1マススライドするという操作仕様を採用しています。この仕様では、キャラクターの移動処理とダイスの移動処理を同時に成立させる必要があり、実装に最も苦労しました。

まず問題になったのは、ダイスが移動した際のキャラクターの挙動です。キャラクターは通常の物理挙動で動作しているため、ダイスだけを移動させるとキャラクターが取り残されたり、逆に物理的な力が加わって不自然に飛んでしまったりすることがありました。また、ダイスの縁での操作判定が曖昧だと、歩いているだけでダイスが動いてしまうなどの誤作動も発生しました。

さらに、ダイスの上から別のダイスへ移動する際にはコリジョンの影響でキャラクターが引っかかったり、ダイスの移動タイミングによってはキャラクターが落下してしまったりと、複数の問題が同時に発生しました。

これらの問題を解決するために、ダイス移動中はキャラクターをダイスに追従させる処理を行い、キャラクターが常にダイスの上に留まるように調整しました。また、ダイス操作については入力を一定時間保持した場合のみスライドを発動するホールド入力方式を採用しました。これに加えて、コリジョンの動的切り替えを実装することにより問題を解決しました。

工夫した点①：マルチプレイを意識した構造

開発当初からマルチプレイ対応を念頭に置き、BoardManager をサーバー権限で管理できる構造にしました。各クライアントからの移動要求を BoardManager が受け取り、盤面状態を一元管理するという設計により、将来的にオンライン対戦モードへの拡張も可能になります。

工夫した点②：Seed 起点方式でゲームバランスを維持

「スポンしたダイスが偶然3つ並んでも消えない」というルールは、ゲームバランスの核となります。このルールを EDiceOrigin による由来管理と Seed 起点探索で実装することで、「プレイヤーが意図した操作だけが結果をもたらす」というパズルゲームとしての面白さを維持しました。

3. 過去の制作作品

スクールでの学習を通じて制作した作品を紹介します。

3.1 Rock Paper Scissors (C++)

ジャンル	コンソールゲーム
使用技術	C++
概要	C++でゲームプログラムの基礎構造（ゲームループ・シーン管理・状態管理）を学ぶ目的で制作したコンソールゲーム。グー・チョキ・パーの三択判定で対戦するじゃんけんゲーム「Rock Paper Scissors」を実装

システム構成

ゲームエンジンの内部構造と同じ設計思想を C++ で実装しました。

RockPaperScissors (ゲーム管理)	シングルトンパターンで実装したゲーム本体クラス。初期化・メインループ・終了処理・シーン管理を担当
SceneBase (基底クラス)	全シーンの共通基底クラス。初期化・更新・終了処理を定義し、派生クラスでオーバーライド
TitleScene / GameScene / ResultScene / EndScene	シーンごとに処理を分離。SceneFactory が State 値を受け取り対応するシーンを生成・遷移を管理
GameData	シーン間で共有するデータの保持。GameScene で生成した数字を ResultScene で参照するデータ共有に使用
Utility (Input / Random / Singleton)	乱数生成・入力処理・シングルトン機能を namespace で共通化

実装した主な機能

- ・ ゲームループ: initialize → mainLoop (終了条件まで繰り返し) → finalize の基本構造を実装
- ・ シーン管理: SceneBase を継承した 4 つのシーンを SceneFactory で生成・切り替え。タイトル→ゲーム→リザルト→終了の遷移を管理
- ・ シングルトンパターン: ゲームインスタンスが同時に 1 つしか存在しない構造を C++ で実装
- ・ 乱数生成: Utility クラスに Random クラスを作成し、グー・チョキ・パーをランダムに選ぶ CPU の手を生成
- ・ 勝敗判定: プレイヤーの入力と CPU の手を比較し、勝ち・負け・あいこの 3 パターンを判定して ResultScene で表示

学んだこと

- ・ ゲームループの基本構造: 実装を通じて、ゲームは初期化・更新・終了というループ構造で動作していることを理解しました。
- ・ シーン管理・状態管理: 場面ごとに処理を分離し、画面遷移の管理を簡潔にするという設計思想を体験しました。
- ・ クラス設計・継承・namespace: C++ のオブジェクト指向の基本を実際のゲーム構造の中で学びました。
- ・ シングルトン: グローバルなゲーム状態を 1 つのインスタンスで管理する設計パターンを体得しました。

3.2 Battery Collector (UE5 + C++)

ジャンル	3D アクション
使用技術	Unreal Engine 5 / C++ + Blueprint
概要	フィールド上にランダムスポーンするバッテリーを集めてエネルギーを維持するゲーム。エネルギーは時間経過で減少し、バッテリー回収で回復。規定値に達するとクリア、0 になるとゲームオーバー

システム構成

C++ でゲームロジック（クラス定義・ルール管理）を実装し、Blueprint で見た目・エフェクト・UI を構築するハイブリッド構成です。

BatteryCollectorGameMode	ゲーム全体の管理。ゲーム状態 (Enum) ・エネルギー減少・勝敗判定・HUD 表示・SpawnVolume 制御
BatteryCollectorCharacter	プレイヤー本体。エネルギー保持・バッテリー回収処理・エネルギー量による見た目変化・移動速度変化
Pickup (親クラス)	拾えるアイテムの基底クラス。StaticMesh・アクティブ状態管理・回収イベントを定義
BatteryPickup (子クラス)	Pickup を継承したバッテリー専用クラス。回復量・物理挙動（落下・転がり）を追加
SpawnVolume	BoxComponent で定義した範囲内にバッテリーをランダム座標・一定間隔で自動生成
BatteryHUD	エネルギーバーと現在値/目標値の表示。ゲーム状態に応じて PLAYING / GAME OVER / GameClear を切り替え

実装した主な機能

- ・ 継承設計: Pickup (基底) → BatteryPickup (派生) の継承構造で、将来の拡張に対応できる設計を実践
- ・ ゲーム状態管理: Playing / GameOver / Won の3状態を Enum で管理。
- ・ Tick 処理による毎フレーム制御: エネルギー減少・勝敗判定を Tick で処理。条件達成時にゲーム状態を切り替え
- ・ SphereComponent による回収範囲: キャラクター周囲の球形範囲内の Pickup を取得し、キーを押すと一括回収
- ・ Dynamic Material Instance: エネルギー量に応じてキャラクターの色をリアルタイムで変化させる視覚的フィードバック
- ・ MaxWalkSpeed 制御: エネルギー残量によって移動速度を変化させるゲームプレイ要素

学んだこと

- ・ C++と Blueprint の役割分担: ロジックは C++、見た目は Blueprint という分担が可能であることを理解しました。
- ・ GameMode によるルール一元管理: ゲームのルール・状態は GameMode が持つという UE の設計思想を体得しました。
- ・ 親クラスに共通処理を集める意義: Pickup / BatteryPickup の継承構造を実装することで、コードの再利用と拡張性の重要性を実感しました。

3.3 マルチプレイ実践 (FPS のネットワーク対応)

使用技術	Unreal Engine 5 / Blueprint
ベース	UE5 付属の FPS サンプルをマルチプレイ対応に改造
概要	ローカルマルチプレイ (同一 PC 上での 2 人プレイ) を動作させ、発生した同期問題を一つひとつ修正。その後 Steam Online Subsystem を使ったオンライン対戦 (セッション作成・参加) まで実装した学習プロジェクト

実装の流れ

STEP 1 ローカルマルチプレイの起動と問題の特定

エディタの Multiplayer Options でプレイヤー数を 2 に設定し、2 クライアントを同時起動。すると 3 つの同期問題が発生しました。

- ・ 問題①: 弾が同期されない (クライアントだけで生成されていたため)
- ・ 問題②: 弾が当たった Cube の動きが同期されない (Replication 未設定)
- ・ 問題③: 発射音が相手クライアントで鳴らない

STEP 2 Replication による同期問題の解決

弾の生成同期	発射処理を「Run On Server」イベントに変更。 Client → Server へ発射を依頼する構造に修正
弾オブジェクトの同期	FirstPersonProjectile_BP の Replicates にチェック。 弾の存在を全クライアントに同期
Cube の移動同期	Replicate Movement を有効化。 物理挙動はサーバーが管理し全クライアントに反映
発射音の同期	音の再生処理を Multicast イベントに変更。 Server → 全クライアントへ通知して再生

STEP 3 Steam Online Subsystem によるオンライン対戦

- ・ DefaultEngine.ini と Plugins 設定で Steam OnlineSubsystem を有効化
- ・ MainMenu レベルを新規作成し、UMG ウィジェットで「Host」と「Client」ボタンを実装
- ・ Host: ホストとしてセッションを作成し、他プレイヤーの接続を待機
- ・ Client: FindSession でセッションを検索し、JoinSession で参加
- ・ FPS 視点対応: FP_Mesh / FP_Gun に Only Owner See を設定し「自分の腕は自分だけに見える」動作を実装

習得したネットワーク同期の基本モデル

Run On Server	クライアントからサーバーへ処理を依頼。ゲームの状態変更はすべてサーバーが行う
Replicates	オブジェクトの存在・状態を全クライアントに同期
Replicate Movement	Actor の位置・回転をネットワーク同期
Multicast	サーバーから全クライアントへ同時通知。音・エフェクトなど演出の同期に使用

この経験が Dice Puzzle の設計に活かしている点

- ・「状態変更はサーバーが行い、クライアントは結果を受け取る」という考え方を体得しました。BoardManager をサーバー権限で一元管理する設計判断の根拠になりました。
- ・Run On Server / Multicast の使い分けが、Dice Puzzle での「盤面ロジック（サーバー側）」と「演出再生（全クライアント）」の責務分離設計と対応しています。

4. 今後の目標

4.1 Dice Puzzle の今後の拡張

- ・ マルチプレイ（オンライン対戦）モードの実装
- ・ マルチプレイ限定の相手への妨害サイコロ（特殊値・特殊テクスチャ）の実装

4.2 さらに学びたい技術

- ・ Blueprint と C++ のハイブリッド開発
- ・ ネットワーク同期・マルチプレイ実装
- ・ シェーダー・マテリアルを駆使した複雑なテクスチャ表現
- ・ ゲームプログラマーとしてのチーム開発技術

4.3 ゲームプログラマーを目指す理由

現在は通信キャリアでエンジニアとして勤務しています。仕事をする中で疑問に思っていた「自分が本当にやりたいことは何か」と向き合い、さまざまなことに挑戦してきました。その中でたどり着いたのがゲーム制作です。「頭の中で想像したものを実現するために、どうすれば良いかを論理的に考えて実行していく」作業に、夢中さと楽しさを見出しました。

ゲーム制作はまさにその連続です。この「作ることが楽しい」という感覚を仕事にしたいと考え、ゲームプログラマーへの転職を決意しました。

モンスターハンターでのマルチプレイの体験、サルゲッチュでの考えさせる設計の面白さ、XI のシンプルなルールの中の深み。プレイヤーとして熱中してきたゲームの面白さを、今度は作る側として実現したいと考えています。